অধ্যায় ১৩

Object-Oriented Programming (OOP): Classes, Prototypes, Inheritance এবং Encapsulation

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) Object ও Class-এর সম্পর্ক, (২) JavaScript-এর Prototype Chain, (৩) Constructor ও `this`, (৪) Inheritance ও Method Overriding, (৫) `#private` Fields দিয়ে Encapsulation, (৬) Composition বনাম Inheritance এবং (৭) বাস্তব Booking Management Application-এর OOP Architecture বুঝতে ও প্রয়োগ করতে পারবে।

১৩.১ OOP কেন শিখব?

একটি Travel Booking System-এ Booking ID, Customer, Price, Status এবং Cancel করার নিয়ম আছে। Flight Booking-এ Baggage Fee, Hotel Booking-এ Number of Nights যোগ হয়। একই বৈশিষ্ট্য ও আচরণ বিভিন্ন ধরনের Booking-এ বারবার লিখলে Duplicate Logic তৈরি হয়। Object-Oriented Programming ডেটা ও সেই ডেটার ওপর কাজ করা Methods-কে একটি নির্দিষ্ট Interface-এর মধ্যে রাখার কৌশল দেয়। তবে OOP সব সমস্যার একমাত্র সমাধান নয়—ছোট Data Transformation-এ সাধারণ Function বেশি সহজ হতে পারে।

চারটি মূল ধারণা: Encapsulation (অভ্যন্তরীণ State নিয়ন্ত্রণ), Abstraction (ব্যবহারকারীর কাছে প্রয়োজনীয় Interface প্রকাশ), Inheritance (একটি Class-এর আচরণ আরেকটি Class-এ গ্রহণ) এবং Polymorphism (একই Interface দিয়ে ভিন্ন আচরণ)। JavaScript-এ এগুলো Prototype-based Object Model-এর ওপর কাজ করে।

১৩.২ Object Literal থেকে Class

```javascript
const booking = {
  id: 'BK-101',
  customer: 'Nadia',
  price: 4500,
  status: 'confirmed',
  cancel() { this.status = 'cancelled'; }
};
booking.cancel();
console.log(booking.status); // cancelled
```

এখানে `this` সাধারণ method call-এর receiver object-কে নির্দেশ করে। কিন্তু Method আলাদা করে Function Variable-এ রাখলে Receiver হারিয়ে যেতে পারে।

```javascript
const person = {
  name: 'Rafi',
  sayName() { return this.name; }
};
console.log(person.sayName()); // Rafi
const detached = person.sayName;
// detached() strict mode-এ this undefined হওয়ায় TypeError হতে পারে
const bound = person.sayName.bind(person);
console.log(bound()); // Rafi
```

`class` ব্যবহার করলে একই Shape-এর একাধিক Object তৈরি সহজ হয়:

```javascript
class Booking {
  constructor(id, price) {
    this.id = id;
    this.price = price;
  }
  summary() { return `${this.id}: BDT ${this.price}`; }
}
const first = new Booking('BK-101', 4500);
const second = new Booking('BK-102', 7200);
console.log(first.summary());
```

`new` নতুন Object তৈরি করে, তার Prototype Link স্থাপন করে এবং Constructor চালায়। Class Constructor-কে `new` ছাড়া সাধারণ Function-এর মতো Call করা যায় না। Class Declaration-ও Declaration-এর আগে ব্যবহার করা নিরাপদ নয়; Temporal Dead Zone-এর নিয়ম প্রযোজ্য।

১৩.৩ Prototype Chain — Class-এর পেছনের মেকানিজম

JavaScript-এর Class Syntax আসলে Prototype-based Inheritance-এর সুবিধাজনক Syntax। Instance-এর Own Property ও Prototype Property আলাদা:

```javascript
class Traveler {
  constructor(name) { this.name = name; }
  greet() { return `Hello, ${this.name}`; }
}
const traveler = new Traveler('Mira');
console.log(Object.hasOwn(traveler, 'name'));  // true
console.log(Object.hasOwn(traveler, 'greet')); // false
console.log(Object.getPrototypeOf(traveler) === Traveler.prototype); // true
```

`greet()` Method প্রতিটি Instance-এর Own Property হিসেবে তৈরি না হয়ে Prototype-এ থাকে। Property Lookup-এর সময় Engine আগে Object, তারপর Prototype Chain পরীক্ষা করে। Chain শেষ হলে `undefined` পাওয়া যায়।

```
traveler instance
  ├── own: name
  └── [[Prototype]] → Traveler.prototype
                        ├── greet()
                        └── [[Prototype]] → Object.prototype → null
```

সতর্কতা: `__proto__` পরিবর্তনের বদলে প্রয়োজন হলে `Object.create()` বা `Object.getPrototypeOf()` ব্যবহার করো। Runtime-এ Prototype পরিবর্তন Performance ও Maintainability ক্ষতিগ্রস্ত করতে পারে।

১৩.৪ Constructor, Instance Methods, Static Methods

JAVASCRIPT

```javascript
class Currency {
  static formatBDT(amount) {
    if (!Number.isFinite(amount)) throw new TypeError('Invalid amount');
    return `BDT ${amount.toFixed(2)}`;
  }
}
console.log(Currency.formatBDT(1500)); // BDT 1500.00
```

`static` Method Class-এর ওপর Call করা হয়, Instance-এর ওপর নয়। Instance State-নির্ভর Method সাধারণ Method হওয়া উচিত। Static Factory Method দিয়ে Named Constructor Pattern তৈরি করা যায়, তবে সব Class-এ Factory প্রয়োজন হয় না।

১৩.৫ Encapsulation ও Private Fields

সাধারণ Public Property বাইরে থেকে পরিবর্তন করা যায়। Booking Price বা Status-এর মতো গুরুত্বপূর্ণ State নিয়ন্ত্রণ করতে Private Fields উপকারী:

JAVASCRIPT

```javascript
class Wallet {
  #balance = 0;
  deposit(amount) {
    if (!Number.isFinite(amount) || amount <= 0) throw new RangeError('Invalid deposit');
    this.#balance += amount;
  }
  get balance() { return this.#balance; }
}
const wallet = new Wallet();
wallet.deposit(500);
console.log(wallet.balance); // 500
// wallet.#balance — SyntaxError: class-এর বাইরে private field access করা যায় না
```

Getter একটি Readable Interface; Setter থাকলে Validation প্রয়োজন। `#private` Fields JavaScript Language-level Privacy দেয়, তবে এটি Authentication, Authorization বা Encryption-এর বিকল্প নয়। Browser-এ থাকা Code ও Data ব্যবহারকারী নিয়ন্ত্রণ করতে পারে।

১৩.৬ Inheritance, `extends`, `super()` ও Overriding

```javascript
class Booking {
  constructor(price) { this.price = price; }
  get total() { return this.price; }
}
class FlightBooking extends Booking {
  constructor(price, baggageFee) {
    super(price); // derived constructor-এ this ব্যবহারের আগে
    this.baggageFee = baggageFee;
  }
  get total() { return super.total + this.baggageFee; }
}
console.log(new FlightBooking(5000, 800).total); // 5800
```

`extends` দিয়ে Parent Class-এর আচরণ পাওয়া যায়। `super()` Parent Constructor চালায়। `super.total` Parent Getter পড়ছে। Overriding-এর মাধ্যমে একই Property/Method Name ভিন্ন Implementation দিতে পারে। Parent-এর Private Field Child সরাসরি Access করতে পারে না; প্রয়োজনীয় Getter/Method দিয়ে Access দিতে হয়।

১৩.৭ Polymorphism ও Open/Closed Design

```javascript
function printTotals(bookings) {
  for (const booking of bookings) {
    console.log(booking.total);
  }
}
```

`printTotals`-কে Flight বা Hotel-এর Internal Calculation জানতে হয় না; `total` Interface-ই যথেষ্ট। এটিই Polymorphism-এর ব্যবহারিক সুবিধা। নতুন Booking Type যোগ করতে Existing Reporting Function পরিবর্তন না-ও লাগতে পারে।

১৩.৮ Composition বনাম Inheritance

সব সম্পর্ক `extends` নয়। "FlightBooking **is a** Booking" যুক্তিযুক্ত; "Booking **has a** NotificationService" হলে Composition উপযুক্ত।

```javascript
class BookingNotifier {
  constructor(sendMessage) { this.sendMessage = sendMessage; }
  notify(booking) { this.sendMessage(`Booking ${booking.id} updated`); }
}
const notifier = new BookingNotifier(console.log);
notifier.notify({ id: 'BK-105' });
```

এভাবে Notification Strategy সহজে Mock করা যায় এবং Booking Domain-কে UI/Email Service থেকে আলাদা রাখা যায়। Inheritance-এর গভীর Chain সাধারণত Avoid করা ভালো।

১৩.৯ Practical Project — Booking Management Lab

আমরা একটি Browser-based Application বানাব, যেখানে Flight ও Hotel Booking একই Manager-এ থাকবে। Source Files:

```
source-code/chapter-13/
├── index.html
├── style.css
├── booking.mjs
├── app.mjs
└── booking.test.mjs
```

Domain Model: `Booking` Base Class, `FlightBooking` ও `HotelBooking` Derived Classes, `BookingManager` Collection ও Operations পরিচালনা করে। UI (`app.mjs`) শুধু Form, DOM ও User Feedback সামলায়। এই Separation-এ Business Logic Browser ছাড়া Test করা যায়।

Feature Rules:

1. Booking ID ও Customer খালি হতে পারবে না; Price অবশ্যই Positive Finite Number।
2. Flight Total = Base Price + Baggage Fee; Fee Negative নয়।
3. Hotel Total = Nightly Price × Number of Nights; Nights Positive Integer।
4. Duplicate Booking ID Reject হবে।
5. Cancelled Booking পুনরায় Cancel করা যাবে না; Confirmed Revenue-তে Cancelled Booking ধরা হবে না।
6. UI-তে User Input `textContent` দিয়ে Render হবে, `innerHTML` দিয়ে নয়।

গুরুত্বপূর্ণ Domain Code

`booking.mjs`-এর প্রধান অংশ:

JAVASCRIPT

```javascript
export class Booking {
  #status = 'confirmed';
  constructor({ id, customer, price }) {
    if (!id?.trim() || !customer?.trim() || !Number.isFinite(price) || price <= 0) {
      throw new Error('Invalid booking');
    }
    this.id = id.trim();
    this.customer = customer.trim();
    this.price = price;
  }
  get status() { return this.#status; }
  get total() { return this.price; }
  cancel() { this.#status = 'cancelled'; }
}
```

এটি **শিক্ষামূলক সংক্ষিপ্ত Illustration**; Companion Source Code-এর `source-code/chapter-13/booking.mjs`-এ আরও কঠোর Validation, Private Fields, Duplicate Handling, Cancellation Guard এবং Serialization আছে।

Run Instructions

Node.js (ES Modules ও Private Fields সমর্থিত সংস্করণ) থাকলে:

```
TERMINAL
cd source-code/chapter-13
node --test booking.test.mjs
```

Browser-এ ES Modules চালাতে Local HTTP Server ব্যবহার করো:

```
TERMINAL
python -m http.server 8000
```

এরপর `http://localhost:8000/` খোলো। Browser UI Manual QA এখনো বাকি। এটি Frontend Learning Demo; Server, Authentication বা Durable Database নেই। Refresh করলে Booking Data Reset হয়।

১৩.১০ Common Mistakes ও Debugging

ভুল	সমস্যা	সমাধান
`new` ছাড়া Class Call	TypeError	`new ClassName()`
Child Constructor-এ `super()`-এর আগে `this`	ReferenceError	আগে `super()`
`this`-সহ Method Detached	Receiver হারায়	`.bind()` বা Arrow Wrapper
`#private` Field বাইরে Access	SyntaxError	Getter/Method ব্যবহার
সব কাজ Inheritance-এ রাখা	Tight Coupling	Composition ও Small Services
Input থেকে Price String রেখেই যোগ	String Concatenation	Explicit Number Conversion + Validation
UI-তে `innerHTML` দিয়ে Customer Name	XSS Risk	`textContent`

১৩.১১ Exercises

1. `Book` Class লিখো, যার `title`, `author` ও `summary()` থাকবে।
2. `Rectangle` Class-এ Validated Width/Height এবং `area` Getter তৈরি করো।
3. `BankAccount`-এ `#balance`, `deposit()` ও `withdraw()` যোগ করো।
4. `Employee` ও `Manager` Inheritance দিয়ে Bonus Calculation করো।
5. `Vehicle` Base Class এবং `Car`, `Bike` Derived Class-এ `describe()` Override করো।
6. `BookingManager`-এ `findByCustomer(name)` যোগ করো (Case-insensitive)।
7. `BookingManager`-এ `confirmedCount` Getter তৈরি করো।

8. Hotel Booking-এ `nights=0` কেন Reject হওয়া উচিত ব্যাখ্যা করো।
9. Composition দিয়ে Booking Confirmation Logger তৈরি করো।
10. `Object.getPrototypeOf()` দিয়ে একটি Class Instance-এর Prototype Chain পরীক্ষা করো।

১৩.১২ Exercise Solutions

JAVASCRIPT

```
// 1
class Book {
  constructor(title, author) { this.title = title; this.author = author; }
  summary() { return `${this.title} — ${this.author}`; }
}
// 2
class Rectangle {
  constructor(width, height) {
    if (![width, height].every(n => Number.isFinite(n) && n > 0)) throw new RangeError('Invalid
size');
    this.width = width; this.height = height;
  }
  get area() { return this.width * this.height; }
}
// 3
class BankAccount {
  #balance = 0;
  get balance() { return this.#balance; }
  deposit(n) { if (!Number.isFinite(n) || n <= 0) throw new RangeError('Invalid'); this.#balance
+= n; }
  withdraw(n) { if (!Number.isFinite(n) || n <= 0 || n > this.#balance) throw new
RangeError('Invalid'); this.#balance -= n; }
}
// 4
class Employee { constructor(salary) { this.salary = salary; } get bonus() { return this.salary
* 0.05; } }
class Manager extends Employee { get bonus() { return this.salary * 0.10; } }
// 5
class Vehicle { describe() { return 'Generic vehicle'; } }
class Car extends Vehicle { describe() { return 'Car'; } }
class Bike extends Vehicle { describe() { return 'Bike'; } }
// 6: BookingManager class-এর ভেতরে যোগ করো:
// findByCustomer(name) { return this.list().filter(b => b.customer.toLocaleLowerCase() ===
name.toLocaleLowerCase()); }
// 7: BookingManager class-এর ভেতরে যোগ করো:
// get confirmedCount() { return this.list({ status: 'confirmed' }).length; }
// 9
class ConfirmationLogger { constructor(log) { this.log = log; } confirmed(b) {
this.log(`Confirmed: ${b.id}`); } }
// 10
const b = new Book('JS', 'Author');
console.log(Object.getPrototypeOf(b) === Book.prototype); // true
```

৮ নম্বরের ব্যাখ্যা: `nights=0` হলে বাস্তব Hotel Stay নেই এবং `total=0` হয়ে যেতে পারে; Domain Validation-এ Positive Integer বাধ্যতামূলক। **৬–৭ নম্বর:** এগুলো Class Body-তে যুক্ত করার Method Snippets, Standalone Script নয়।

১৩.১৩ Self-Assessment Quiz (১০টি MCQ)

1. Class-এর Instance তৈরি করতে সাধারণত কোন Keyword? (A) `create` (B) `new` (C) `instance` (D) `make`
2. Class Method সাধারণত কোথায় থাকে? (A) Global Object (B) Prototype (C) JSON (D) DOM
3. Private Field Declaration কোনটি? (A) `private x` (B) `_x` (C) `#x` (D) `@x`
4. Derived Constructor-এ Parent Constructor চালাতে কী লাগে? (A) `parent()` (B) `base()` (C) `super()` (D) `this()`
5. `static` Method কীভাবে Call করা হয়? (A) Class-এর ওপর (B) সবসময় Instance-এর ওপর (C) `window` দিয়ে (D) `new` দিয়ে
6. একই `total` Interface-এ ভিন্ন Booking Calculation কোন ধারণা? (A) Hoisting (B) Polymorphism (C) Closure (D) Event Loop
7. `Object.hasOwn(obj, 'x')` কী পরীক্ষা করে? (A) Prototype-এ x আছে কি না (B) Own Property আছে কি না (C) x Function কি না (D) x Private কি না
8. একটি Booking-এর মধ্যে Logger Inject করা কোন Design? (A) Composition (B) Inheritance (C) Recursion (D) Mutation
9. Cancelled Booking Revenue থেকে বাদ দেওয়ার Logic কোথায় থাকা ভালো? (A) CSS (B) Domain/Manager (C) HTML (D) Browser URL
10. User Input DOM-এ Text হিসেবে Render করতে কোনটি নিরাপদ Default? (A) `innerHTML` (B) `document.write()` (C) `textContent` (D) `eval()`

Quiz Answers ও Explanations

Q	Answer	Explanation
1	B	`new` Class Constructor দিয়ে Instance তৈরি করে।
2	B	সাধারণ Instance Methods Prototype-এ থাকে।
3	C	`#` Language-level Private Field Syntax।
4	C	`super()` Parent Constructor চালায়।
5	A	Static Method Class Constructor-এর Property।
6	B	একই Interface ভিন্ন Object Type-এ ভিন্নভাবে কাজ করে।
7	B	Own Property পরীক্ষা করে, Inherited নয়।
8	A	Logger Dependency Inject করা Composition।
9	B	Domain Rules UI থেকে স্বাধীন রাখা ভালো।
10	C	`textContent` Text হিসেবে Render করে; HTML Parse করে না।

১৩.১৪ আরও অনুশীলন ও Production Checklist

- Booking-এ `createdAt` এবং `updatedAt` যোগ করো।

- Booking ID Random Generation-এর Collision Risk ব্যাখ্যা করো।
- `BookingManager`-এর Data Storage Adapter আলাদা করে Unit Test করো।
- Money Calculation-এ Floating-Point সমস্যা এড়াতে Minor Unit (Paisa) বা Decimal Library-এর প্রয়োজন বিবেচনা করো।
- Real Application-এ Server-side Validation, Authentication, Authorization, Database Transaction এবং Audit Log প্রয়োজন।

Chapter Summary: JavaScript-এর OOP হলো Prototype-based System-এর ওপর গড়ে ওঠা Class, Encapsulation, Inheritance ও Polymorphism-এর ব্যবহারিক প্রয়োগ। Domain Rules এবং UI আলাদা করলে Testing ও Maintenance সহজ হয়। পরের অধ্যায়ে Advanced Functions, Closures এবং Functional Patterns শিখব।